



UFRJ



INSTITUTO DE
COMPUTAÇÃO
UFRJ

Programação de Computadores II

Laboratório - Métodos de Busca

Prof. Marcos Tomazzoli Leipnitz

Exercício 1

- Altere a função de busca sequencial, versão que lida com elementos do arranjo armazenados em ordem crescente, para lidar com elementos repetidos. Sendo: **void buscaSeqOrdRep(int arr[], int tam, int el, int *pos, int *quant)**, onde ***pos** indica a posição onde o primeiro elemento igual a **el** se encontra no arranjo **arr** de tamanho **tam**; o número de ocorrências de elemento **el** deve ser indicado em ***quant**. Caso o elemento **el** não esteja presente no arranjo **arr**, a função deve indicar posição (***pos**) igual a -1 e o número de ocorrências (***quant**) igual a 0.
-

Exercício 1

```
int buscaSeqOrd(int arr[], int
tam, int el) {
    int i;
    for (i=0; i<tam; i++) {
        if (arr[i] >= el) {
            if (arr[i] == el)
                return i;
            else return -1; }
    }
    return -1; }
```

Exercício 1

- Faça um programa principal para testar a função implementada. A seguir, um exemplo da formatação desejada para apresentação de entradas e saídas.

Entrada	Saída
Digite a quantidade de elementos do vetor: 8	Posição da
Digite os elementos do vetor: 1 1 3 3 3 4 4 5	primeira
Digite o elemento procurado: 3	ocorrência: 2
	Quantidade de
	ocorrências: 3

Exercício 2

- Altere o método de busca binária para que, a cada divisão do vetor, ele imprima quantos elementos foram descartados na busca.
- Faça um programa principal para testar a função implementada.

Entrada	Saída
Digite a quantidade de elementos do vetor: 5	1a. iteração: 3
Digite os elementos do vetor: 1 2 3 4 5	2a. iteração: 1
Digite o elemento procurado: 1	

Exercício 2

```
int buscaBin(int arr[], int tam, int el){
    int fim = tam-1;
    int ini = 0;
    while (ini <= fim) {
        int meio = (fim + ini)/2;
        if (arr[meio] < el)
            ini = meio + 1;
        else {
            if (arr[meio] > el)
                fim = meio - 1;
            else return meio;
        }
    }
    return -1; }
```

Exercício 3

- Analise a função a seguir:

```
int descubra(int vet[], int n, int k) {  
    int x=0, y=0;  
    for(int i=0; i<n; i++){  
        if(vet[i]<k) //comparacao 1  
            x+=vet[i];  
        if(vet[i]>k) //comparaçao 2  
            y+=vet[i];  
    }  
    return y-x; }
```

- a) O que essa função retorna?
- b) Quantas vezes a comparação 1 é realizada?
- c) Quantas vezes a comparação 2 é realizada?
- d) Qual é o total de comparações?

Exercício 4

- A função a seguir implementa a versão recursiva da busca binária. Complete os valores das lacunas.

```
bool buscaR(int v[], int n, int procurado, int esq, int dir)
{
    int meio = (esq+dir)/2;
    if(esq<=dir){
        if(v[meio] == procurado)
            return true;
        else if(v[meio] > procurado)
            return buscaR(v, n, procurado, _____, _____);
        else if(v[meio] < procurado)
            return buscaR(v, n, procurado, _____, _____);
    }
    return false; }
```

- Crie um programa principal para testar a função implementada.

Desafio 8

- Faça a função `void buscaMenorMaiorBin(int arr[], int tam, int el, int *menor, int *maior)` que, dado um arranjo `arr` ordenado (alocado dinamicamente e acessado através de aritmética de ponteiros) com `tam` números inteiros distintos e um inteiro `el`, atualize o número de elementos do arranjo que são menores do que `el` em `*menor` e também o número de elementos que são maiores do que `el` em `*maior`. Sua proposta de solução deve ser construída usando busca binária.
- Faça um programa principal para testar a função implementada, conforme exemplos a seguir.

Desafio 8

- Exemplos de Entrada e Saída em Tela

Entrada	Saída
7 20 1 3 4 5 7 9 10	Menor: 7 – Maior: 0
7 5 1 3 4 5 7 9 10	Menor: 3 – Maior: 3
7 1 1 3 4 5 7 9 10	Menor: 0 – Maior: 6
7 6 1 3 4 5 7 9 10	Menor: 4 – Maior: 3
8 20 1 3 4 5 7 9 10 15	Menor: 8 – Maior: 0